

Systems Engineering of LLM Workstation

Kirill Nevzorov under supervision
of Prof. Christopher Buckley
University of Sussex
School of Engineering and Informatics

March 29, 2026

Abstract

This report presents the design and implementation of an LLM-powered workstation application built with a Quarkus backend and a web frontend. Structured analysis of local project artifacts through a tool-calling workflow is created in which an LLM orchestrates exploration, reading, and summarization operations. Rather than treating LLM assistance as ad hoc code generation, the project applies conventional software engineering practices, including modular architecture, explicit interfaces, validation checks, and iterative refinement. The report documents the runtime architecture for tool dispatch, alias resolution, transaction-scoped execution, and read-context persistence, with attention to safety controls such as path validation and constrained filesystem traversal. Development and evaluation were conducted in a home-lab setting using off-the-shelf models and standard engineering tooling. The findings highlight practical trade-offs between development speed, maintainability, and robustness when integrating LLMs into software engineering workflows.

Table of Contents

1. Introduction
2. Java with Quarkus - Overview
3. Coding with LLMs - Background
4. LLM Powered Software - Software Architecture
5. Project Description
6. Project Overview
7. Engineering at Home
8. Results and Discussion

1. Introduction

This section provides an overview of the project, its objectives, and its significance in the field of LLM-powered software development.

Objectives

To explore the integration of LLMs in software architecture

To demonstrate the practical implementation of LLM-assisted development practices

To analyze the challenges and benefits of using LLMs in engineering workflows

Significance

The project contributes to the growing body of research on LLM-powered development by providing a concrete example of how these technologies can be applied in real-world engineering scenarios.

TODO: Bring In From 92 Percent Report

TODO: Add a concise project overview paragraph mirroring the old report's "Project Overview" framing but adapted to LLM workstation goals.

TODO: Add a dedicated motivation subsection explaining why LLM tool-calling was chosen over conventional implementation-only workflows.

TODO: Add a short "Findings and Achievements" preview subsection listing 3-5 concrete outcomes that will be evidenced later.

TODO: Add a "Professional Considerations" subsection covering reproducibility, responsible AI use, and evaluation constraints.

2. Java with Quarkus - Overview

What is Quarkus?

Quarkus is a Kubernetes-native Java framework that provides a set of features optimized for containerized applications. It is built on top of the Java Virtual Machine (JVM) and is designed to deliver fast startup times and low memory footprint, making it ideal for microservices and cloud-native applications.

Why Quarkus for This Project?

Performance: Quarkus' GraalVM native image support allows our application to run efficiently in containerized environments.

Developer Productivity: The framework's built-in features for REST APIs, CDI, and JPA reduce boilerplate code and accelerate development.

Integration with LLMs: Quarkus' lightweight architecture makes it easier to integrate with LLM-powered components while maintaining scalability.

Key Features Utilized

RESTEasy: For building API endpoints that interact with the LLM backend.

SmallRye Config: For managing configuration settings in a Kubernetes environment.

Panache: Simplifies database interactions with minimal boilerplate code.

TODO: Bring In From 92 Percent Report

TODO: Add a "Requirement Analysis" subsection for the Quarkus stack decision.

TODO: Split requirements into "Functional" and "Non-Functional" lists using the same structure as the source report.

TODO: Add a "Deviations" subsection describing how this project departs from a standard Quarkus CRUD architecture.

TODO: Add one table mapping requirements to concrete implementation evidence in this codebase.

3. Coding with LLMs - Background

The Role of LLMs in Software Development

Large Language Models (LLMs) have revolutionized software development by enabling tasks such as code generation, bug detection, documentation, and natural language to code translation. Their ability to understand and generate human-like text has made them invaluable tools for modern engineering workflows.

Key Capabilities of LLMs in Coding

Code Generation: LLMs can produce functional code snippets or entire modules based on natural language descriptions.

Debugging Assistance: They can identify potential bugs, suggest fixes, and explain error messages.

Documentation: LLMs automate the creation of API documentation, comments, and technical write-ups.

Refactoring: They assist in restructuring code for improved readability and maintainability.

Integration with Engineering Workflows

LLMs are increasingly being integrated into development pipelines through tools like GitHub Copilot, which provides real-time suggestions during coding. This report explores how such technologies enhance productivity while maintaining traditional software engineering principles.

TODO: Bring In From 92 Percent Report

TODO: Add an explicit "Introduction" subsection that states complexity and scope assumptions.

TODO: Add a subsection analogous to "Unity" that explains the selected LLM tooling ecosystem and why it was practical.

TODO: Add one "Techniques" subsection documenting the most complex implemented workflow (for this project: incremental tool use orchestration).

TODO: Add a short subsection documenting one unresolved technical issue and why it was deferred due to time.

4. LLM Powered Software - Software Architecture

System Overview

The software architecture uses a layered tool-calling model for LLM-driven engineering workflows. The implementation combines a Quarkus backend, modular tool-dispatch components, and a React-based frontend. The design prioritizes maintainability, safety controls, and predictable runtime behavior under iterative LLM interaction.

Architectural Pattern

The runtime follows a modular command-dispatch architecture:

1. A service-level dispatcher selects a module that can handle a requested tool.
2. Each module delegates execution to a registry of single-purpose tool macros.

3. Each macro validates arguments and performs one cohesive behavior.

This combines plugin-style extensibility at module level with command-style isolation at tool-operation level.

Core Components

Tool service entry point with transactional execution boundaries

Tool modules for capability grouping (explore, read, write, introspect)

Macro registry for canonical-name and alias resolution

Tool macro implementations as single-responsibility operations

Read-context persistence support for caching discovered project knowledge

Execution Flow

Given a tool execution request, the backend:

1. Selects the first module that can handle the tool name.
2. Parses JSON tool arguments.
3. Resolves aliases/keyphrases to canonical tool names.
4. Executes the matching macro.
5. Returns a string result to the calling LLM runtime.

Naming and Compatibility

Canonical tool names are used internally, while aliases and legacy keyphrases are retained for backward compatibility with older prompts and conversation state. Name resolution is normalized through lowercase/trim preprocessing before lookup.

Error and Failure Model

The error model is intentionally explicit and LLM-readable:

Unknown tool errors for unsupported names

Invalid argument errors for parse failures

Tool execution failures for runtime exceptions

Tools may additionally return domain-specific messages for path validation and file-access failures.

Persistence and Context Caching

Tool execution is transaction-scoped, and read-oriented operations persist context into project knowledge storage. Cached entries include file contents, path summaries, detailed path analyses, and folder snapshots. Cache persistence is best-effort so the primary tool response is preserved even when cache writes fail.

Security and Robustness Controls

Path traversal protection for project-scoped operations

Directory existence/type validation before recursive traversal

Ignore rules for high-cost trees such as .git, node_modules, target, and build

Output truncation and sampling to limit context volume and token overhead

Integration with Frontend

The architecture exposes REST endpoints used by the frontend for file navigation, content retrieval, chat operations, and runtime activity inspection. This separation keeps UI orchestration independent from backend tool-dispatch internals.

Design Trade-Offs

The macro-per-operation design increases class count but improves local testability and safer refactoring. String-oriented tool input/output is practical for LLM interaction, though less type-safe for non-LLM clients.

TODO: Bring In From 92 Percent Report

TODO: Add a dedicated subsection on event-driven coordination analogous to the source report's event-system discussion.

TODO: Add an explicit "Inversion of Control" subsection that describes dependency-injection constraints in this project context.

TODO: Add one diagram showing component/module communication and where coupling is intentionally accepted.

TODO: Add an "Alternatives" subsection comparing current architecture with at least two credible alternatives and rationale for rejection.

5. Project Description

Project Objectives

This project aims to: - Develop a LLM-powered web application that analyzes file system contents using Meta's Llama 3.2 model - Demonstrate the integration of LLMs with Java-based microservices using Quarkus - Explore the practical implementation of LLM-assisted development practices in real-world engineering scenarios

Scope

The project focuses on: - Building a web interface for file system analysis - Implementing LLM-driven content interpretation - Creating a scalable architecture for LLM integration - Documenting the development workflow and challenges

Methodology

We employed a structured approach combining traditional software engineering practices with LLM-assisted development: 1. Requirements analysis and system design 2. LLM-assisted code generation for core components 3. Integration testing and iterative refinement 4. Documentation of the development process and outcomes

TODO: Bring In From 92 Percent Report

TODO: Add a "Basic Workflow" subsection equivalent to the source report's gameplay loop, but for user-to-tool-to-output flow.

TODO: Add one "Highlight" subsection for the most advanced implemented feature (tool-calling architecture).

TODO: Add one "Performance Problem" subsection documenting the bottleneck observed in practice.

TODO: Add at least one figure reference or screenshot callout to support the narrative.

6. Project Overview

This project develops an LLM-powered software system using a Java/Quarkus backend and a web-based frontend. The engineering emphasis is on reliable tool-calling workflows where an LLM can issue structured operations against project files, inspect outputs, and iteratively refine implementation decisions.

Scope

The implemented scope covers:

Tool registration and dispatch

Per-tool macro execution model

Alias-to-canonical name resolution

Error and transaction behavior

Read-context caching into persistent project knowledge

Frontend integration for file exploration and chat interaction

System Modules

The backend groups functionality into modules:

Explore: filesystem discovery, navigation, and path analysis

Read: file and folder content reads

Write: controlled output generation and persistence support

Introspect: session and project manifest views

Each module contributes tool macros that implement single operations with explicit contracts.

Delivery Model

Development follows an iterative LLM-assisted workflow:

1. Define constraints and expected tool behavior.
2. Implement or refine focused tool macros.
3. Validate through tests and runtime checks.
4. Persist useful read-context for later tool calls.
5. Expose stable backend interfaces to the frontend.

Expected Outcomes

The architecture is intended to deliver:

Deterministic tool dispatch under mixed alias usage

Safer refactoring through low-coupling tool classes

Improved interaction efficiency through context caching

Clear integration boundaries between backend and frontend layers

TODO: Bring In From 92 Percent Report

TODO: Add a subsection listing major development challenges and how they were mitigated.

TODO: Add a subsection on experiment or iteration method (what changed between versions and why).

TODO: Add one subsection that records negative results or dead ends, not only successful decisions.

7. Engineering at Home

This section documents the local engineering workflow used to develop and evaluate the project in a home-lab environment.

Local Stack

Java backend with Quarkus

React 18 + TypeScript frontend

Vite 5 for frontend build and development tooling

Markdown rendering via react-markdown and remark-gfm

Development Workflow

Typical local workflow:

1. Start backend services.
2. Run frontend development mode.
3. Test filesystem navigation and chat tool-calling flows.
4. Validate endpoint contracts and timeout behavior.
5. Build production assets and re-test integration.

Frontend Application Structure

The frontend is organized into:

Entry point and application shell

Reusable visual components

Stateful hooks for menu panels and file highlighting

API helper layer for typed GET/POST calls

Shared type definitions and theme styles

Primary UI Areas

Top menu and modal panels for runtime configuration

Current working directory navigation

File browser panel

Assistant chat panel

Explorer canvas with floating project windows

Backend API Usage

Core endpoint groups exercised in local testing:

Filesystem and project endpoints (cwd, directory listing, tree, file, raw file)

Assistant endpoints (chat, one-shot requests, reset)

Activity endpoints (tool activity stream)

Configuration endpoints (runtime menu settings)

Practical Constraints in Home Setup

Long assistant requests require higher timeout budgets than standard API calls.

Large directories must be filtered to avoid expensive scans.

Local resource limits can affect responsiveness during concurrent tool execution.

Strict path validation is required to prevent accidental traversal outside project scope.

Operational Conventions

Maintain explicit loading and error states in UI flows.

Keep API responses strongly typed.

Avoid path guessing in frontend logic; rely on backend-provided data.

Use dedicated long-timeout paths for assistant calls.

TODO: Bring In From 92 Percent Report

TODO: Add a subsection quantifying hardware and compute limits encountered during development.

TODO: Add a subsection discussing local-vs-cloud trade-offs and the cost constraints that affected choices.

TODO: Add concrete examples where limited compute changed the experimental plan or scope.

8. Results and Discussion

Key Findings

The implementation produced several concrete engineering outcomes:

Deterministic tool dispatch through module selection and canonical-name resolution

Backward-compatible invocation through alias/keyphrase mapping

Transaction-scoped execution for tool operations that involve persistence

Best-effort read-context caching that preserves user-facing results even on cache-write failure

Frontend/backend separation where UI orchestration remains independent of dispatch internals

Discussion

The tool-macro architecture improved maintainability by enforcing single-responsibility operations and reducing mutation hotspots associated with monolithic switch-based dispatch. The trade-off is increased class count and greater need for naming discipline across modules. For LLM interaction, string-oriented tool interfaces improved practical usability, although stronger typed schemas would benefit non-LLM clients.

Limitations

First-match module dispatch makes ordering significant if duplicate tool names are introduced.

String-oriented inputs and outputs reduce type safety for programmatic downstream consumers.

Cache invalidation is limited, so stale snapshots remain possible without explicit refresh.

Home-lab resource constraints can affect consistency of perceived responsiveness.

Validation Evidence

Architecture claims are supported by toolset-level and integration-level tests, including read-context persistence checks that assert project-knowledge writes after tool invocation. This supports evidence-based claims for dispatch determinism, alias compatibility, and persistence side effects.

Future Work

Future research could explore: 1. Introducing typed tool responses alongside current LLM-friendly string outputs 2. Strengthening cache invalidation and freshness policies 3. Extending observability with per-tool latency and failure-rate metrics 4. Evaluating alternative dispatch strategies when module overlap increases

TODO: Bring In From 92 Percent Report

TODO: Add an explicit “Overview” subsection before results, matching the chapter pattern used in the source PDF.

TODO: Add one grounded performance subsection that separates observed evidence from expectations.

TODO: Add a future-work subsection for architecture optimization analogous to the source report’s algorithm replacement plan.

TODO: Add a future-work subsection for model/agent improvement analogous to the source report’s alternative-learning-method discussion.

TODO: Add one paragraph on what the project did not prove yet, to keep claims calibrated.